Machine Learning for Sound and Music
Lab 1: Music Genre Classification

Lidia Wiśniewska

October 2026

This report summarizes the work presented in the Jupyter notebooks.

1 Introduction

Music genre classification can help organize large music collections and support applications such as music search and recommendation. In this task, machine learning models are trained to predict a song's genre from audio descriptors. The aim is to compare several classification models and identify a configuration with high classification accuracy.

2 Task 1

2.1 Data Analysis

The analysis begins with exploratory data analysis and basic data cleaning. The `GenreAll.csv` file initially contains 993 songs, 240 feature columns, and one target column, `genre`. No missing values were found.

Duplicate rows were checked using the feature columns, excluding the genre label. There were 14 pairs of duplicate feature rows. One pair had conflicting genre labels. Since the audio could not be checked manually, both rows in that pair were removed. For each of the other 13 pairs, one duplicate row was removed. This left 978 songs in the cleaned dataset.

The feature columns include numerical and categorical data. The numerical features were examined using summary statistics and distribution plots presented below (Figure 1). Their scales and distributions vary, and some are skewed. Additionally, two numerical features, `spectral_barkbands_25_var` and `spectral_barkbands_25_dvar`, had zero variance and were removed because of their redundancy.

Four features were also randomly selected for closer inspection against the genre labels (Figure 2). The notebook contains the detailed observations from these plots. Correlations between numerical features were explored by heatmaps, including a view of highly correlated features with $|r| > 0.9$ and a heatmap of

the full feature set. These plots were used to understand redundancy in the data – 64 features are correlated above the set threshold (0.9). The model pipelines perform the correlation filtering during training.

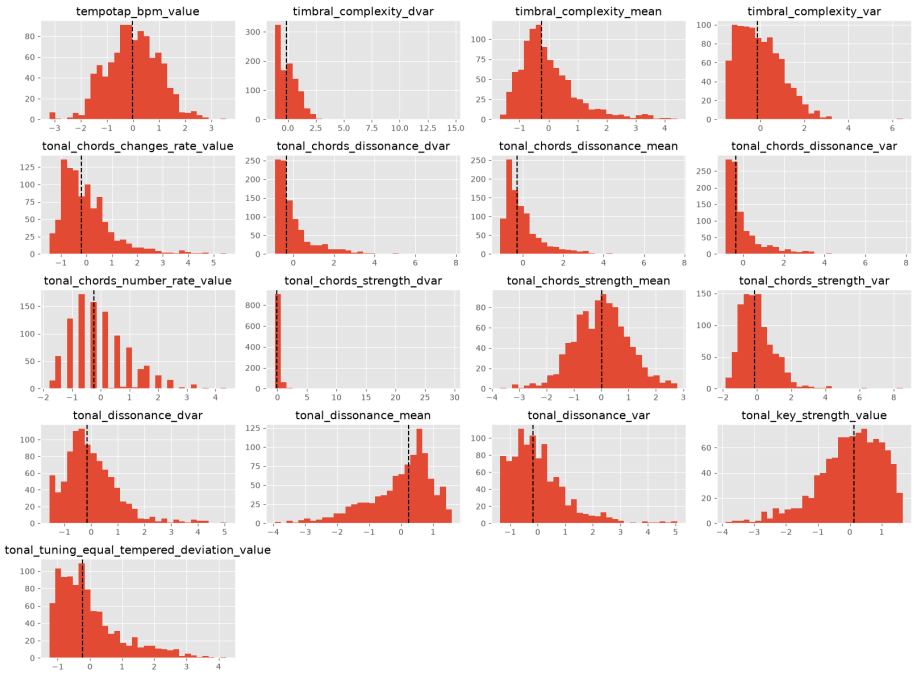


Figure 1: Distributions of the last 17 numerical features.

There are three binary categorical descriptors in addition to the target column. Their relationship with genre was examined using cross-tabulations (Figure 3). For example, the feature `highlevel_voice_instrumental_value` appears to separate some pop songs from instrumental tracks, although this feature alone is not enough to classify all genres. The genre distribution is slightly imbalanced (Figure 4), so class weighting is used for some of the models.

2.2 Feature Engineering

Preprocessing is placed inside scikit-learn pipelines so that each transformation is fitted using only the training data available to that fold. This is especially important during cross-validation, because it prevents information from a validation fold from influencing the preprocessing fitted on its corresponding training fold.

A `ColumnTransformer` applies `RobustScaler` to numerical features and `OrdinalEncoder` to the binary categorical descriptors. The target column is excluded from the input features. Robust scaling uses statistics that are less sensitive to outliers than the mean and standard deviation.

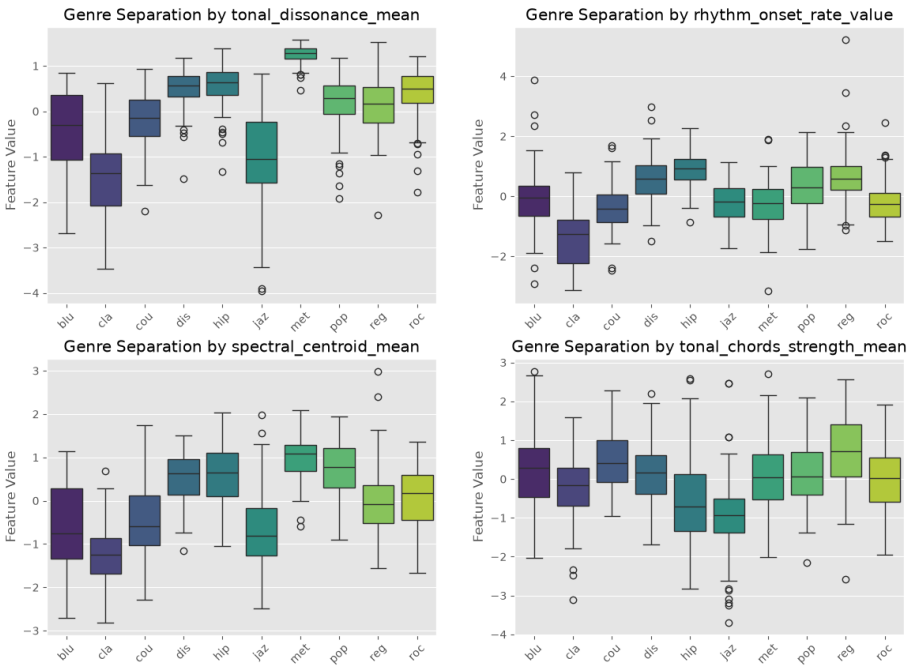


Figure 2: Genre separation for four selected features.

A custom correlation filter then removes redundant features. It calculates the absolute feature correlations and, for each feature in column order, drops it if its correlation with any previously retained feature is greater than 0.9. The customization comes from initial idea of comparing features with the target when deciding which feature to retain. However, it did not yield better results and remained as classic correlation filter.

Two preprocessing pipelines are used:

1. `preprocessor_1`, used by Logistic Regression, kNN and MLP:
   (a) scale numerical features and encode categorical features;
   (b) remove highly correlated features;
   (c) use `SelectKBest` with `f_classif` to select features.
2. `preprocessor_2`, used by Decision Tree and Random Forest:
   (a) scale numerical features and encode categorical features.

For Logistic Regression and kNN, the number of selected features is tuned as part of the parameter search. Decision Tree and Random Forest use their tree-based split mechanisms rather than a separate `SelectKBest` step. The MLP receives all features produced by `preprocessor_2`; it does not perform explicit feature selection in this pipeline.

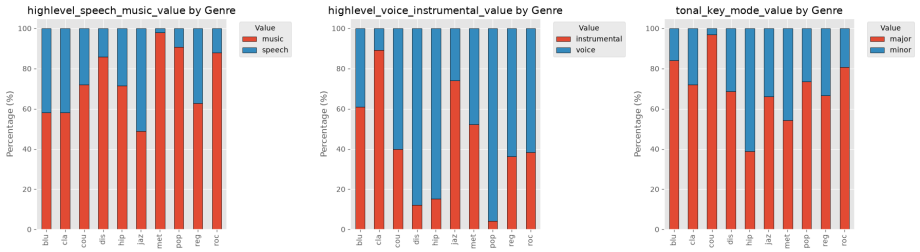


Figure 3: Cross-tabulations of categorical features with the genre label.

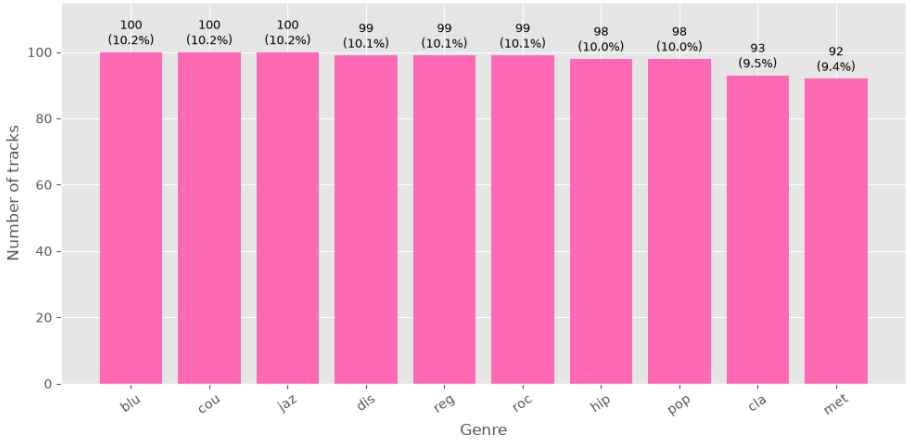


Figure 4: Distribution of the target genre classes.

2.3 Training and Hyperparameter Tuning

Five classifiers are compared:

- Logistic Regression
- k-nearest neighbors (kNN)
- Decision Tree
- Random Forest
- Multilayer Perceptron (MLP)

The target genre labels are encoded separately from the input features using `LabelEncoder`. The data is then divided into a stratified training and test split, using 80% of the songs for training and 20% for testing. Stratification preserves approximately the same genre proportions in both subsets.

Each model is placed after its relevant preprocessing pipeline. Hyperparameters are tuned with `GridSearchCV`, using 10-fold stratified cross-validation on

the training split and accuracy as the scoring metric. The configuration with the highest mean validation accuracy is selected. With `refit=True`, which is the `GridSearchCV` default, the selected configuration is then refitted on the complete training split and ready for final evaluation on the test set. The parameter grids are:

1. Logistic Regression
   - Number of features selected by `SelectKBest`: [100, 120, 140, `all`]
   - Inverse regularization strength C: [0.5, 0.7, 1.0, 1.5]
   - Regularization mix `l1_ratio`: [0.0, 1.0]
     - 0.0 corresponds to L2 regularization.
     - 1.0 corresponds to L1 regularization.
2. kNN
   - Number of features selected by `SelectKBest`: [60, 100, 140, `all`]
   - Number of neighbors: [3, 5, 7, 9, 11, 15, 21]
   - Weighting: [`uniform`, `distance`]
   - Minkowski distance parameter p: [1, 2]
     - $p = 1$: Manhattan distance.
     - $p = 2$: Euclidean distance.
3. Decision Tree
   - Split criterion: [`gini`, `entropy`]
   - Maximum depth: [`None`, 5, 10, 20]
   - Minimum samples per leaf: [1, 2, 5] `SelectKBest` step.
4. Random Forest
   - Number of trees: [100, 300]
   - Maximum tree depth: [`None`, 10, 20]
   - Minimum samples per leaf: [1, 2]
   - Features considered at each split: [`sqrt`, 0.5]
5. MLP
   - Hidden-layer sizes: [`(64,)`, `(128,)`, `(64, 32)`, `(128, 64, 32)`]
   - L2 regularization strength α: [0.001, 0.01]
   - Initial learning rate: [0.01, 0.05, 0.1]
   - The MLP receives all features from `preprocessor_2`; there is no separate feature-selection step.

Logistic Regression, Decision Tree, and Random Forest use balanced class weights. The MLP uses balanced sample weights. kNN does not use class weighting or oversampling. The MLP also uses early stopping with a maximum of 500 iterations. The tuned hyperparameters and feature selection thresholds (k) for each model are as follows:

1. Logistic Regression: C = 0.7, l1_ratio = 1.0, feature_selection_k = 120
2. kNN: metric = minkowski, n_neighbors = 7, p = 1, weights = distance, feature_selection_k = 100
3. Decision Tree: ccp_alpha = 0.0, criterion = entropy, max_depth = None, min_samples_leaf = 1
4. Random Forest: ccp_alpha = 0.0, max_depth = None, max_features = 0.5, min_samples_leaf = 1, n_estimators = 100
5. MLP: alpha = 0.001, hidden_layer_sizes = (128, 64), learning_rate_init = 0.01, feature_selection_k = 140

2.4 Evaluation

Each model was tuned using 10-fold stratified cross-validation on the training portion of a single stratified train/test split (random seed 42). The best configuration for each model was refitted on the complete training portion and evaluated on the same held-out test set. Table 1 shows the mean training and validation accuracies for the selected configuration during cross-validation, together with its test accuracy.

Table 1: Cross-validation and test accuracy for each model.

Model	CV train accuracy	CV validation accuracy	Test accuracy
Logistic Regression	0.967	**0.840**	**0.872**
MLP	0.984	0.817	0.837
Random Forest	1.000	0.795	0.842
kNN	1.000	0.757	0.765
Decision Tree	1.000	0.643	0.653

Logistic Regression achieved the highest mean cross-validation validation accuracy (84%) and also the highest test accuracy (87.2%) on this split. The Random Forest performed reasonably well, reaching a test accuracy of 84.2%.

The Decision Tree and Random Forest both had a cross-validation training accuracy of 100%, while their validation accuracies were much lower. This gap suggests that both tree-based models overfit the training folds. For both models, the tuning search selected `max_depth=None` and `min_samples_leaf=1`; for cost-complexity pruning, it selected `ccp_alpha=0.0`, meaning that the tested pruning options did not improve the mean validation accuracy. Further investigation of pruning and tree complexity was not carried out because of time constraints.

The test accuracies were measured on only one held-out split with seed 42. Therefore, they describe performance on this particular split and do not show how much the results vary across different splits. This should be further tested, yet was not carried out because of time constraints.

3 Task 2

3.1 Data Analysis

Task 2 uses two versions of the music dataset: F30, containing one feature vector for each 30-second song, and F3, containing feature vectors for 3-second excerpts. Both datasets contain 57 numerical predictors and a genre label. The F3 filenames encode both the source song and the fragment number, which allows clips to be associated with their original songs. No missing feature values were found.

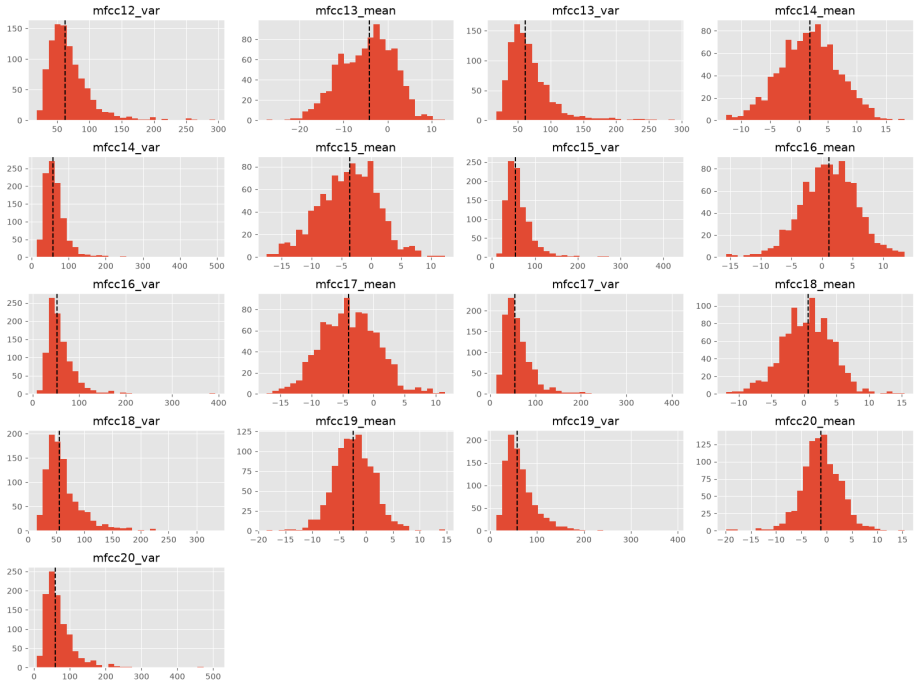


Figure 5: Distributions of selected numerical features for 30-second songs (F30).

The distributions of selected numerical features are shown in Figures 5 and 6. In particular, the variance features are strongly right-skewed. Across the 28 variance features, the mean skewness was 2.15; applying a logarithmic transformation reduced it to 0.36. The feature distributions also have substantially

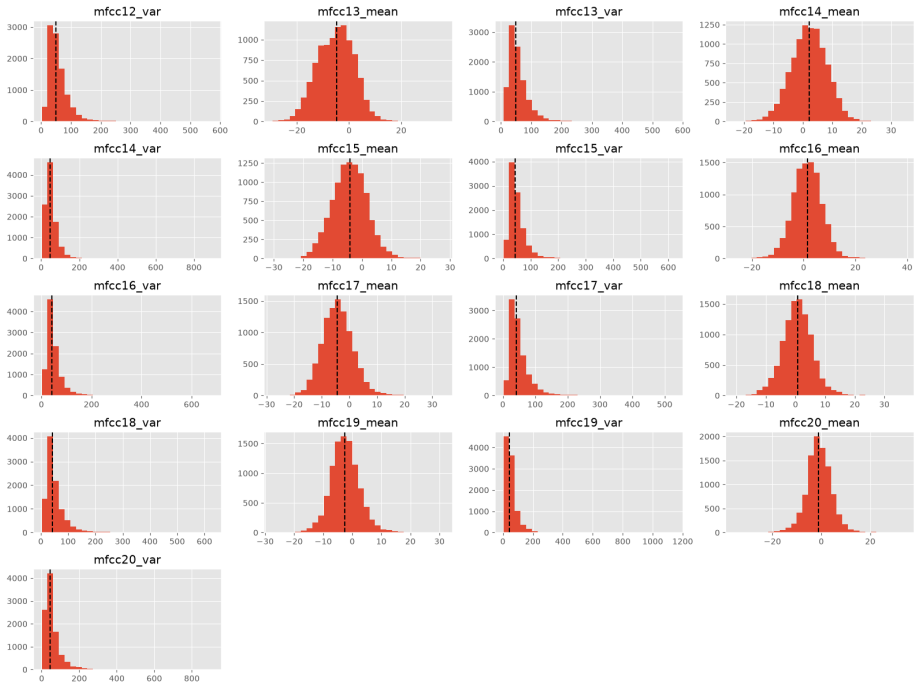


Figure 6: Distributions of selected numerical features for 3-second fragments (F3).

different scales, which is relevant for models that rely on distances or gradient-based optimization.

Figures 7 and 8 show how selected features vary across genres. Although some features exhibit differences between genres, the distributions overlap, indicating that no single feature is sufficient to distinguish all classes. The genre distributions after cleaning are shown in Figures 9 and 10. The classes remain reasonably balanced, with between 91 and 100 songs per genre.

3.2 Data Preprocessing

Before training, exact duplicate feature vectors were identified using the audio features rather than the filenames. Fourteen pairs of identical tracks were found. For pairs with the same genre label, one copy was retained. The pair `metal.00058` and `rock.00016` had conflicting genre labels, so both tracks were removed. The F3 dataset also contained three shared clips between `reggae.00073` and `reggae.00074`; these tracks were retained but assigned to the same group to prevent their clips from appearing in different splits. In total, 15 song IDs were removed, leaving 985 F30 songs and 9,840 F3 clips associated with 984 song groups.

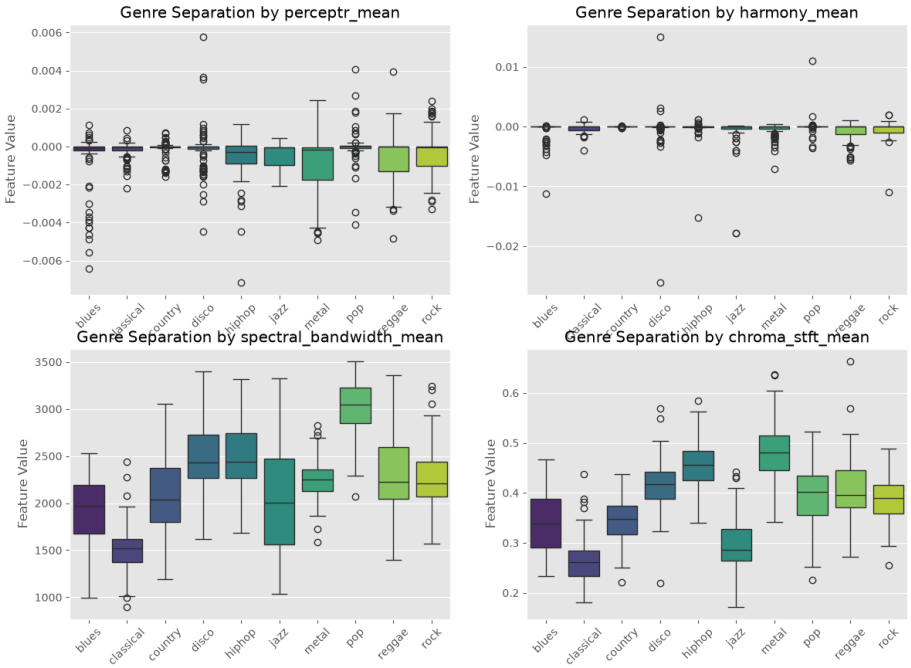


Figure 7: Genre separation for selected features in F30.

Identifiers, group information, and the genre label were not included among the predictors. The same song-level outer split was used for both datasets: 788 training songs and 197 held-out test songs. This corresponds to 7,871 training clips and 1,969 test clips in F3. Ten stratified group folds were then created from the training songs and reused for both datasets. Thus, clips from a song could not appear in both a training and validation fold, or in both the training and test sets.

For scale-sensitive models, preprocessing was performed within each training fold. The 28 variance features were transformed using $\log(1 + x)$, followed by standardization. A correlation filter at $|r| > 0.8$ was investigated but not included in the final pipelines: it removed 14 of the 57 predictors, including all roll-off and zero-crossing-rate features, and reduced predictive performance. Feature selection using again the `SelectKBest` with f-scores was included as a tunable option in processing pipeline, those were prepared similarly as in task 1: Two preprocessing pipelines are used:

1. `preprocessor_1`, used by Logistic Regression, kNN, SVM, MLP:
   (a) log-transform the variances;
   (b) scale numerical features with `StandardScaler`
   (c) use `SelectKBest` with `f_classif` to select features.

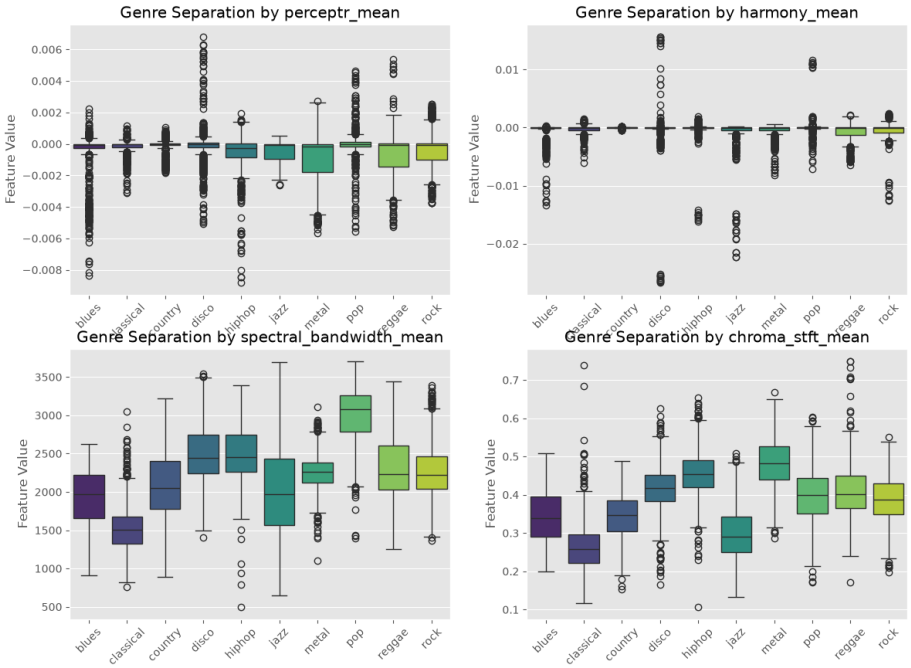


Figure 8: Genre separation for selected features in F3.

2. `preprocessor_2`, used by Decision Tree, Random Forest, HistGradient-Boosting:
   (a) log-transform the variances;
   (b) scale numerical features with `StandardScaler`.

3.3 Training and Hyperparameter Tuning

For both datasets seven classifiers were trained separately:

- Logistic Regression
- k-nearest neighbors (kNN)
- Decision Tree
- Random Forest
- an RBF-kernel Support Vector Machine (SVM)
- Histogram-based Gradient Boosting (HistGradientBoosting)
- Multilayer Perceptron (MLP)

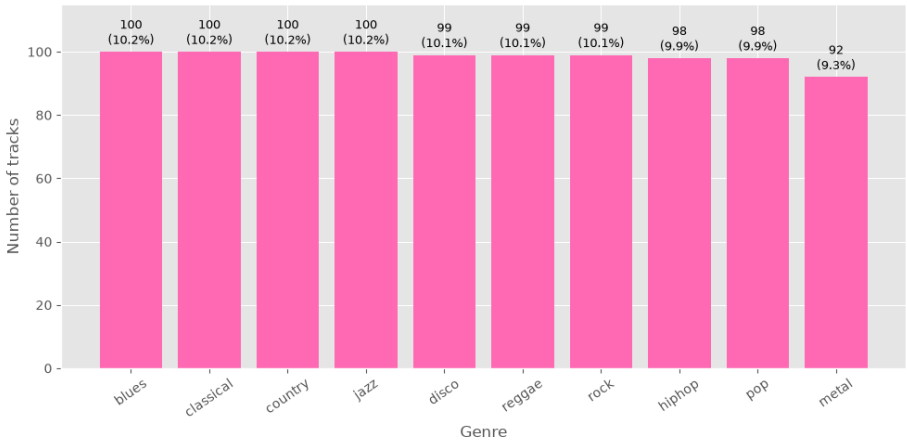


Figure 9: Genre distribution in F30.

Each model and dataset combination was tuned independently using grid search and the same ten stratified, song-grouped folds. Accuracy was used for model selection, and all preprocessing and feature selection steps were fitted within the cross-validation pipeline. The grids were adjusted based on the performance in task 1 and were following:

1. Logistic Regression
   - Number of features selected by `SelectKBest`: $[10, 20, 30, 45, \texttt{all}]$
   - Inverse regularization strength C: $[0.03, 0.1, 0.3, 1.0]$
2. k-nearest Neighbors (kNN)
   - Number of features selected by `SelectKBest`: $[15, 30, \texttt{all}]$
   - Number of neighbors: $[5, 9, 15, 25]$
   - Minkowski distance parameter p: $[1, 2]$
     - $p = 1$: Manhattan distance.
     - $p = 2$: Euclidean distance.
   - Neighbor weighting was fixed to `distance`.
3. Decision Tree
   - Split criterion: [`gini`, `entropy`]
   - Maximum depth: $[8, 12, \texttt{None}]$
   - Minimum samples per leaf: $[1, 5, 10]$

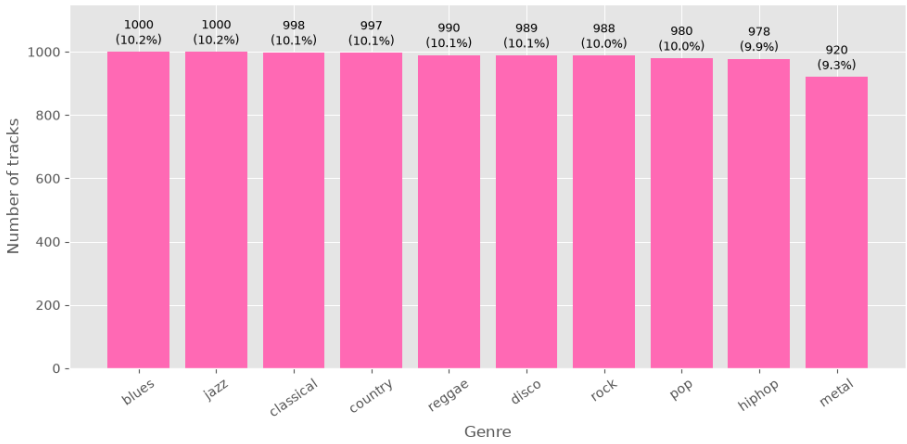


Figure 10: Genre distribution in F3.

4. Random Forest
   - Features considered at each split: [`sqrt`, 0.3]
   - Minimum samples per leaf: [1, 3]
   - Number of trees was fixed at 300.
5. Histogram-based Gradient Boosting
   - Learning rate: [0.05, 0.1]
   - Maximum number of leaf nodes: [15, 31]
   - Maximum number of iterations was fixed at 300.
6. Support Vector Machine (RBF kernel)
   - Inverse regularization strength C: [1, 3, 10, 30]
   - The kernel and γ were fixed to RBF and `scale`, respectively.
7. Multilayer Perceptron (MLP)
   - Hidden-layer sizes: [`(64, 32)`, `(128, 64)`, `(256, 128)`]
   - L2 regularization strength α: [0.3, 1.0, 3.0]

Logistic Regression, Decision Tree, Random Forest, and the RBF SVM used balanced class weights. kNN used distance weighting and no class weighting. SelectKBest was tuned for Logistic Regression and kNN; for the other models using the same preprocessing pipeline, its feature count remained at the default `all`. Tree-based models used the separate preprocessing pipeline without feature selection.

The best parameters selected by cross-validation were:

1. Logistic Regression:
   - F30: $C = 0.3,\ k = \texttt{all}$
   - F3: $C = 0.1,\ k = \texttt{all}$
2. kNN:
   - F30: $n_{neighbors} = 5,\ p = 1,\ k = \texttt{all}$
   - F3: $n_{neighbors} = 15,\ p = 1,\ k = \texttt{all}$
3. Decision Tree:
   - F30: $criterion = \texttt{entropy},\ max_depth = 8,\ min_samples_leaf = 1$
   - F3: $criterion = \texttt{entropy},\ max_depth = 12,\ min_samples_leaf = 1$
4. Random Forest:
   - F30: $max_features = 0.3,\ min_samples_leaf = 1$
   - F3: $max_features = 0.3,\ min_samples_leaf = 1$
5. Histogram-based Gradient Boosting:
   - F30: $learning_rate = 0.05,\ max_leaf_nodes = 15$
   - F3: $learning_rate = 0.1,\ max_leaf_nodes = 31$
6. RBF SVM:
   - F30: $C = 10$
   - F3: $C = 3$
7. MLP:
   - F30: $\alpha = 3.0,\ hidden_layer_sizes = \texttt{(64, 32)}$
   - F3: $\alpha = 1.0,\ hidden_layer_sizes = \texttt{(256, 128)}$

The F30 models were trained using full-song feature vectors. The F3 models were trained using individual 3-second clips. Although both datasets use the same held-out set of source songs, their evaluation units differ: F30 accuracy is calculated per song, whereas F3 accuracy is calculated per clip.

Table 2: Cross-validation and test accuracy for each model

Dataset	Model	CV train accuracy	CV validation accuracy	Test accuracy
F30	SVM	1.000	$\mathbf{0.755 \pm 0.031}$	**0.741**
F30	MLP	0.969	0.750 ± 0.041	0.695
F30	HistGradientBoosting	1.000	0.731 ± 0.037	0.716
F30	Logistic regression	0.877	0.718 ± 0.053	0.716
F30	Random forest	1.000	0.714 ± 0.045	0.650
F30	kNN	1.000	0.704 ± 0.047	0.650
F30	Decision tree	0.809	0.511 ± 0.042	0.487
F3	HistGradientBoosting	1.000	$\mathbf{0.741 \pm 0.019}$	**0.732**
F3	SVM	0.989	0.731 ± 0.022	0.706
F3	MLP	0.983	0.720 ± 0.030	0.714
F3	Random forest	1.000	0.700 ± 0.020	0.669
F3	kNN	1.000	0.678 ± 0.031	0.656
F3	Logistic regression	0.754	0.670 ± 0.033	0.670
F3	Decision tree	0.954	0.520 ± 0.024	0.478

3.4 Evaluation

Table 2 reports the cross-validation training accuracy, mean validation accuracy and its standard deviation, and held-out test accuracy. The F30 test results are song-level measurements; the F3 test results are clip-level measurements and should not be interpreted as directly equivalent to song-level accuracy.

The best cross-validation result for F30 was obtained by the SVM, with a validation accuracy of 0.755 and a held-out song accuracy of 0.741. For F3, HistGradientBoosting achieved the highest validation accuracy, 0.739, and a held-out clip accuracy of 0.734. These results are based on one fixed, stratified outer split with random seed 42, so they may vary with a different split.

3.5 Research Questions

Is the 3 s accuracy real or inflated? It is inflated if split randomly. A random clip cross-validation split allowed clips from the same source song to appear in both training and validation folds. For HistGradientBoosting, this produced an accuracy of 0.931, compared with 0.739 using the song-grouped folds, an increase of 0.192. The corresponding random-split and grouped-split accuracies were 0.892 and 0.678 for kNN, and 0.729 and 0.670 for logistic regression. This comparison demonstrates why clips from the same song must be kept together during splitting.

Which songs are misclassified by every model? On F30, 17 of the 197 held-out songs were misclassified by all seven models. The F3 analysis also identified songs with many misclassified clips; 11 songs that were consistently misclassified at the F30 level also had at least five clips that all seven F3 models classified incorrectly. The errors were therefore concentrated in some source songs rather than being distributed uniformly across the test set.

Which feature families matter most? Grouped permutation importance for the best F30 model (SVM) showed the largest accuracy decrease when MFCC means were permuted (0.337), followed by MFCC variances (0.264) and spectral-shape features (0.101). For the best F3 model (HistGradientBoosting), MFCC means were again most important (0.254), followed by harmonic/percussive features (0.195) and MFCC variances (0.107). These values are exploratory, test-set permutation-importance results: they indicate predictive contribution for the fitted models, not causal effects.

Does feature selection reduce overfitting or just cost? For logistic regression and kNN, selecting fewer features reduced the train–validation accuracy gap, but also reduced validation accuracy. For example, on F30 logistic regression achieved validation accuracy 0.718 using all 57 features, compared with 0.713 using 45 features; on F3, the corresponding values were 0.670 and 0.661. kNN also achieved its highest validation accuracy with all 57 features on both datasets. In these experiments, feature selection reduced the apparent overfitting gap but did not improve validation accuracy; the best settings retained all features.

4 Conclusions

This report explored music genre classification using various machine learning models on both full-song (30-second) with general Essentia descriptors and the provided feature set, and clip-level (3-second) audio features. In Task 1, Logistic Regression achieved the highest test accuracy (87.2%), while tree-based models such as Decision Trees and Random Forests exhibited significant overfitting.

In Task 2, the RBF SVM performed best on the 30-second dataset (74.1% test accuracy), whereas Histogram-based Gradient Boosting achieved the highest accuracy on the 3-second clips (73.2%). The analysis highlighted the critical importance of grouped cross-validation; failing to isolate 3-second clips from the same source song resulted in severely inflated performance metrics due to data leakage.

Furthermore, permutation importance analysis revealed that MFCC means and variances were the strongest predictors of music genre across the top-performing models. Finally, while explicit feature selection slightly reduced the train–validation gap, it did not improve overall validation accuracy, demonstrating that the models benefited most from retaining the complete set of predictive features.